

Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations

Maziar Raissi¹, Paris Perdikaris², and George Em Karniadakis¹

¹*Division of Applied Mathematics, Brown University,
Providence, RI, 02912, USA*

²*Department of Mechanical Engineering and Applied Mechanics,
University of Pennsylvania,
Philadelphia, PA, 19104, USA*

Abstract

We introduce *physics informed neural networks* – neural networks that are trained to solve supervised learning tasks while respecting any given law of physics described by general nonlinear partial differential equations. In this two part treatise, we present our developments in the context of solving two main classes of problems: data-driven solution and data-driven discovery of partial differential equations. Depending on the nature and arrangement of the available data, we devise two distinct classes of algorithms, namely continuous time and discrete time models. The resulting neural networks form a new class of *data-efficient* universal function approximators that naturally encode any underlying physical laws as prior information. In this first part, we demonstrate how these networks can be used to infer solutions to partial differential equations, and obtain physics-informed surrogate models that are fully differentiable with respect to all input coordinates and free parameters.

Keywords: Data-driven scientific computing, Machine learning, Predictive modeling, Runge-Kutta methods, Nonlinear dynamics

1. Introduction

With the explosive growth of available data and computing resources, recent advances in machine learning and data analytics have yielded transformative results across diverse scientific disciplines, including image recognition [1], natural language processing [2], cognitive science [3], and genomics [4].

物理驱动深度学习（第一部分）：非线性偏微分方程的数据驱动解法

Maziar Raissi¹, Paris Perdikaris², 和 George Em Karniadakis¹

¹*Division of Applied Mathematics, Brown University,
Providence, RI, 02912, USA*

²*Department of Mechanical Engineering and Applied Mechanics,
University of Pennsylvania,
Philadelphia, PA, 19104, USA*

Abstract

我们介绍了 *physics informed neural networks* —— 这种神经网络经过训练，可以在遵循由一般非线性偏微分方程描述的任何物理定律的前提下，解决监督学习任务。在这篇两部分的论文中，我们展示了在解决两类主要问题的背景下我们的研究进展：数据驱动的求解和数据驱动的偏微分方程发现。根据可用数据的性质和排列，我们设计了两类不同的算法，即连续时间模型和离散时间模型。由此产生的神经网络形成了一类新的 *data-efficient* 通用函数逼近器，自然地将任何潜在的物理定律作为先验信息进行编码。在第一部分中，我们展示了如何使用这些网络来推断偏微分方程的解，并获得物理信息驱动的代理模型，这些模型对于所有输入坐标和自由参数都是完全可微的。

Keywords: 数据驱动的科学计算，机器学习，预测建模，龙格-库塔方法，非线性动力学

1. Introduction

随着可用数据和计算资源的爆炸性增长，近年来机器学习和数据分析的进展在包括图像识别[1]、自然语言处理[2]、认知科学[3]和基因组学[4]在内的各个科学领域取得了变革性的成果。

However, more often than not, in the course of analyzing complex physical, biological or engineering systems, the cost of data acquisition is prohibitive, and we are inevitably faced with the challenge of drawing conclusions and making decisions under partial information. In this *small data* regime, the vast majority of state-of-the art machine learning techniques (e.g., deep/-convolutional/recurrent neural networks) are lacking robustness and fail to provide any guarantees of convergence.

At first sight, the task of training a deep learning algorithm to accurately identify a nonlinear map from a few – potentially very high-dimensional – input and output data pairs seems at best naive. Coming to our rescue, for many cases pertaining to the modeling of physical and biological systems, there exist a vast amount of prior knowledge that is currently not being utilized in modern machine learning practice. Let it be the principled physical laws that govern the time-dependent dynamics of a system, or some empirical validated rules or other domain expertise, this prior information can act as a regularization agent that constrains the space of admissible solutions to a manageable size (for e.g., in incompressible fluid dynamics problems by discarding any non realistic flow solutions that violate the conservation of mass principle). In return, encoding such structured information into a learning algorithm results in amplifying the information content of the data that the algorithm sees, enabling it to quickly steer itself towards the right solution and generalize well even when only a few training examples are available.

The first glimpses of promise for exploiting structured prior information to construct data-efficient and physics-informed learning machines have already been showcased in the recent studies of [5, 6, 7]. There, the authors employed Gaussian process regression [8] to devise functional representations that are tailored to a given linear operator, and were able to accurately infer solutions and provide uncertainty estimates for several prototype problems in mathematical physics. Extensions to nonlinear problems were proposed in subsequent studies by Raissi *et. al.* [9, 10] in the context of both inference and systems identification. Despite the flexibility and mathematical elegance of Gaussian processes in encoding prior information, the treatment of nonlinear problems introduces two important limitations. First, in [9, 10] the authors had to locally linearize any nonlinear terms in time, thus limiting the applicability of the proposed methods to discrete-time domains and compromising the accuracy of their predictions in strongly nonlinear regimes.

然而，在分析复杂的物理、生物或工程系统的过程中，数据获取的成本往往是高昂的，我们不可避免地面临在部分信息下做出结论和决策的挑战。在这种 *small data* 情况下，绝大多数最先进的机器学习技术（例如深度/卷积/递归神经网络）缺乏鲁棒性，并且无法提供任何收敛保证。

乍一看，训练一个深度学习算法以准确识别非线性映射，从少量——可能是非常高维——的输入输出数据对出发，似乎充其量是幼稚的。在许多涉及物理和生物系统建模的情况下，有一个大量的先验知识可以帮助我们，而这些在现代机器学习实践中尚未被充分利用。无论是支配系统时变动力学的基本物理定律，还是一些经验验证的规则或其他领域专长，这些先验信息都可以作为一种正则化手段，将可接受解的空间约束到一个可管理的大小（例如，在不可压缩流体动力学问题中，通过舍弃任何违反质量守恒原理的不现实流动解）。作为回报，将这种结构化信息编码到学习算法中，可以增强算法所使用数据的信息含量。

在最近的研究[5, 6, 7]中，已经展示了利用结构化先验信息构建数据高效且物理启发的学习机器的初步前景。在这些研究中，作者采用高斯过程回归[8]设计了针对给定线性算子量身定制的函数表示，并能够准确推断多个数学物理原型问题的解，并提供不确定性估计。随后，Raissi *et. al.* [9, 10]在推断和系统识别的背景下提出了对非线性问题的扩展。尽管高斯过程在编码先验信息方面具有灵活性和数学优雅性，但处理非线性问题引入了两个重要限制。首先，在[9, 10]中，作者必须在时间上对任何非线性项进行局部线性化，从而限制了所提出方法在离散时间域的适用性，并影响了a

Secondly, the Bayesian nature of Gaussian process regression requires certain prior assumptions that may limit the representation capacity of the model and give rise to robustness/brittleness issues, especially for nonlinear problems [11].

1.1. Problem setup and summary of contributions

In this work we take a different approach by employing deep neural networks and leverage their well known capability as universal function approximators [12]. In this setting, we can directly tackle nonlinear problems without the need for committing to any prior assumptions, linearization, or local time-stepping. We exploit recent developments in automatic differentiation [13] – one of the most useful but perhaps underused techniques in scientific computing – to differentiate neural networks with respect to their input coordinates and model parameters to obtain *physics informed neural networks*. Such neural networks are constrained to respect any symmetry, invariance, or conservation principles originating from the physical laws that govern the observed data, as modeled by general time-dependent and nonlinear partial differential equations. This simple yet powerful construction allows us to tackle a wide range of problems in computational science and introduces a potentially disruptive technology leading to the development of new data-efficient and physics-informed learning machines, new classes of numerical solvers for partial differential equations, as well as new data-driven approaches for model inversion and systems identification.

The general aim of this work is to set the foundations for a new paradigm in modeling and computation that enriches deep learning with the long-standing developments in mathematical physics. These developments are presented in the context of two main problem classes: data-driven solution and data-driven discovery of partial differential equations. To this end, let us consider parametrized and nonlinear partial differential equations of the general form

$$u_t + \mathcal{N}[u; \lambda] = 0,$$

where $u(t, x)$ denotes the latent (hidden) solution and $\mathcal{N}[\cdot; \lambda]$ is a nonlinear operator parametrized by λ . This setup encapsulates a wide range of problems in mathematical physics including conservation laws, diffusion processes, advection-diffusion-reaction systems, and kinetic equations. As a motivating example, the one dimensional Burgers' equation [14] corresponds to the case

其次，高斯过程回归的贝叶斯性质需要某些先验假设，这可能会限制模型的表示能力，并引发鲁棒性/脆弱性问题，尤其是对于非线性问题 [11]。

1.1. Problem setup and summary of contributions

在这项工作中，我们采取了一种不同的方法，通过使用深度神经网络并利用它们作为通用函数逼近器的广为人知的能力 [12]。在这种情况下，我们可以直接处理非线性问题，而无需对任何先验假设进行承诺、线性化或局部时间步进。我们利用了自动微分的最新发展 [13] —— 这是科学计算中最有用但可能未被充分使用的技术之一 —— 来对神经网络进行输入坐标和模型参数的微分以获得 *physics informed neural networks*。这样的神经网络被约束以遵守任何由支配观测数据的物理定律所产生的对称性、不变性或守恒原理，这些定律由一般的时间相关非线性偏微分方程建模。这种简单而强大的构建使我们能够处理计算科学中的广泛问题，并引入一种可能具有颠覆性的技术，导致

这项工作的总体目标是为建模和计算中新范式奠定基础，该范式将深度学习与数学物理中长期发展的成果相结合。这些发展是在两个主要问题类别的背景下提出的：数据驱动的求解和偏微分方程的数据驱动发现。为此，我们考虑一般形式的参数化非线性偏微分方程。

$$u_t + \mathcal{N}[u; \lambda] = 0,$$

其中 $u(t, x)$ 表示潜在的（隐藏的）解， $\mathcal{N}[\cdot; \lambda]$ 是由 λ 参数化的非线性算子。这一设置涵盖了数学物理中的广泛问题，包括守恒定律、扩散过程、平流-扩散-反应系统以及动力学方程。作为一个激励性例子，一维 Burgers 方程 [14] 对应于这种情形

where $\mathcal{N}[u; \lambda] = \lambda_1 u u_x - \lambda_2 u_{xx}$ and $\lambda = (\lambda_1, \lambda_2)$. Here, the subscripts denote partial differentiation in either time or space. Given noisy measurements of the system, we are interested in the solution of two distinct problems. The first problem is that of predictive inference, filtering and smoothing, or data driven solutions of partial differential equations [9, 5] which states: given fixed model parameters λ what can be said about the unknown hidden state $u(t, x)$ of the system? The second problem is that of learning, system identification, or data-driven discovery of partial differential equations [10, 6, 15] stating: what are the parameters λ that best describe the observed data?

In this first part of our two-part treatise, we focus on computing data-driven solutions to partial differential equations of the general form

$$u_t + \mathcal{N}[u] = 0, \quad x \in \Omega, \quad t \in [0, T], \quad (1)$$

where $u(t, x)$ denotes the latent (hidden) solution, $\mathcal{N}[\cdot]$ is a nonlinear differential operator, and Ω is a subset of $\mathbb{R}^D$. In what follows, we put forth two distinct classes of algorithms, namely continuous and discrete time models, and highlight their properties and performance through the lens of different benchmark problems. All code and data-sets accompanying this manuscript are available at <https://github.com/maziarraissi/PINNs>.

2. Continuous Time Models

We define $f(t, x)$ to be given by the left-hand-side of equation (1); i.e.,

$$f := u_t + \mathcal{N}[u], \quad (2)$$

and proceed by approximating $u(t, x)$ by a deep neural network. This assumption along with equation (2) result in a *physics informed neural network* $f(t, x)$. This network can be derived by applying the chain rule for differentiating compositions of functions using automatic differentiation [13].

2.1. Example (Burgers' Equation)

As an example, let us consider the Burgers' equation. This equation arises in various areas of applied mathematics, including fluid mechanics, nonlinear acoustics, gas dynamics, and traffic flow [14]. It is a fundamental partial differential equation and can be derived from the Navier-Stokes equations for the velocity field by dropping the pressure gradient term. For small values of

其中 $\mathcal{N}[u; \lambda] = \lambda_1 u u_x - \lambda_2 u_{xx}$ 和 $\lambda = (\lambda_1, \lambda_2)$ 。这里，下标表示时间或空间上的偏微分。给定系统的噪声测量，我们关注两个不同问题的解。第一个问题是预测推断、滤波和平滑，或偏微分方程的基于数据的解 [9, 5]，其陈述为：给定固定的模型参数 λ ，关于系统的未知隐状态 $u(t, x)$ 可以说些什么？第二个问题是学习、系统识别或偏微分方程的基于数据的发现 [10, 6, 15]，其陈述为：哪些参数 λ 能最好地描述观测数据？

在我们两部分论文的第一部分中，我们专注于计算通用形式的偏微分方程的数据驱动解法

$$u_t + \mathcal{N}[u] = 0, \quad x \in \Omega, \quad t \in [0, T], \quad (1)$$

其中 $u(t, x)$ 表示潜在（隐藏）解， $\mathcal{N}[\cdot]$ 是非线性微分算子， Ω 是 $\mathbb{R}^D$ 的一个子集。接下来，我们提出两类不同的算法，即连续时间模型和离散时间模型，并通过不同基准问题的视角强调它们的特性和性能。随本文稿附带的所有代码和数据集可在 <https://github.com/maziarraissi/PINNs> 获取。

2. Continuous Time Models

我们将 $f(t, x)$ 定义为由方程 (1) 的左边给出；即，

$$f := u_t + \mathcal{N}[u], \quad (2)$$

并通过用深度神经网络近似 $u(t, x)$ 来继续。这个假设以及方程 (2) 导致了一个 *physics informed neural network* $f(t, x)$ 。这个网络可以通过使用自动微分 [13] 对函数复合进行链式求导来推导。

2.1. Example (Burgers' Equation)

作为一个例子，让我们考虑伯格方程。这个方程出现在应用数学的各个领域，包括流体力学、非线性声学、气体动力学和交通流[14]。它是一个基本的偏微分方程，并且可以通过从速度场的纳维-斯托克斯方程中去掉压力梯度项推导出来。对于小的值

the viscosity parameters, Burgers' equation can lead to shock formation that is notoriously hard to resolve by classical numerical methods. In one space dimension, the Burger's equation along with Dirichlet boundary conditions reads as

$$\begin{aligned} u_t + uu_x - (0.01/\pi)u_{xx} &= 0, \quad x \in [-1, 1], \quad t \in [0, 1], \\ u(0, x) &= -\sin(\pi x), \\ u(t, -1) &= u(t, 1) = 0. \end{aligned} \quad (3)$$

Let us define $f(t, x)$ to be given by

$$f := u_t + uu_x - (0.01/\pi)u_{xx},$$

and proceed by approximating $u(t, x)$ by a deep neural network. To highlight the simplicity in implementing this idea we have included a Python code snippet using Tensorflow [16]; currently one of the most popular and well documented open source libraries for machine learning computations. To this end, $u(t, x)$ can be simply defined as

```
def u(t, x):
    u = neural_net(tf.concat([t,x],1), weights, biases)
    return u
```

Correspondingly, the *physics informed neural network* $f(t, x)$ takes the form

```
def f(t, x):
    u = u(t, x)
    u_t = tf.gradients(u, t)[0]
    u_x = tf.gradients(u, x)[0]
    u_xx = tf.gradients(u_x, x)[0]
    f = u_t + u*u_x - (0.01/tf.pi)*u_xx
    return f
```

The shared parameters between the neural networks $u(t, x)$ and $f(t, x)$ can be learned by minimizing the mean squared error loss

$$MSE = MSE_u + MSE_f, \quad (4)$$

粘度参数下，Burgers方程可能会导致冲击形成，而传统数值方法 notoriously 难以解决。在一个空间维度下，Burgers方程以及Dirichlet边界条件表示为

$$\begin{aligned} u_t + uu_x - (0.01/\pi)u_{xx} &= 0, \quad x \in [-1, 1], \quad t \in [0, 1], \\ u(0, x) &= -\sin(\pi x), \\ u(t, -1) &= u(t, 1) = 0. \end{aligned} \quad (3)$$

我们定义 $f(t, x)$ 为

$$f := u_t + uu_x - (0.01/\pi)u_{xx},$$

并通过用神经网络逼近 $u(t, x)$ 来继续。为了突出实现这一想法的简便性，我们包含了一个使用 TensorFlow [16] 的 Python 代码片段；TensorFlow 目前是最流行且文档最完善的开源机器学习计算库之一。为此， $u(t, x)$ 可以简单地定义为

```
def u(t, x):
    u = neural_net(tf.concat([t,x],1), weights, biases)
    return u
```

相应地，*physics informed neural network* $f(t, x)$ 采用以下形式

```
def f(t, x):
    u = u(t, x)
    u_t = tf.gradients(u, t)[0]
    u_x = tf.gradients(u, x)[0]
    u_xx = tf.gradients(u_x, x)[0]
    f = u_t + u*u_x - (0.01/tf.pi)*u_xx
    return f
```

神经网络 $u(t, x)$ 和 $f(t, x)$ 之间共享的参数可以通过最小化均方误差损失来学习

$$MSE = MSE_u + MSE_f, \quad (4)$$

where

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2,$$

and

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$

Here, $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ denote the initial and boundary training data on $u(t, x)$ and $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ specify the collocations points for $f(t, x)$. The loss MSE_u corresponds to the initial and boundary data while MSE_f enforces the structure imposed by equation (3) at a finite set of collocation points.

In all benchmarks considered in this work, the total number of training data N_u is relatively small (a few hundred up to a few thousand points), and we chose to optimize all loss functions using L-BFGS; a quasi-Newton, full-batch gradient-based optimization algorithm [17]. For larger data-sets a more computationally efficient mini-batch setting can be readily employed using stochastic gradient descent and its modern variants [18, 19]. Despite the fact that there is no theoretical guarantee that this procedure converges to a global minimum, our empirical evidence indicates that, if the given partial differential equation is well-posed and its solution is unique, our method is capable of achieving good prediction accuracy given a sufficiently expressive neural network architecture and a sufficient number of collocation points N_f . This general observation deeply relates to the resulting optimization landscape induced by the mean square error loss of equation 4, and defines an open question for research that is in sync with recent theoretical developments in deep learning [20, 21]. Here, we will test the robustness of the proposed methodology using a series of systematic sensitivity studies that accompany the numerical results presented in the following.

Figure 1 summarizes our results for the data-driven solution of the Burgers equation. Specifically, given a set of $N_u = 100$ randomly distributed initial and boundary data, we learn the latent solution $u(t, x)$ by training all 3021 parameters of a 9-layer deep neural network using the mean squared error loss of (4). Each hidden layer contained 20 neurons and a hyperbolic tangent activation function. In general, the neural network should be given sufficient approximation capacity in order to accommodate the anticipated

哪里

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2,$$

和

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$

这里, $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ 表示 $u(t, x)$ 上的初始和边界训练数据, $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ 指定了 $f(t, x)$ 的配点。损失 MSE_u 对应于初始和边界数据, 而 MSE_f 在有限数量的配点上强制由方程 (3) 施加的结构。

在本文考虑的所有基准中, 训练数据的总数 N_u 相对较少 (从几百到几千个点不等), 我们选择使用 L-BFGS 优化所有损失函数; 这是一种拟牛顿、全批量基于梯度的优化算法 [17]。对于更大的数据集, 可以使用随机梯度下降及其现代变体 [18, 19] 轻松采用计算效率更高的小批量设置。尽管没有理论保证该过程会收敛到全局最小值, 但我们的实证结果表明, 如果给定的偏微分方程是良定的且其解是唯一的, 在具有足够表达能力的神经网络架构和足够多的配点 N_f 的情况下, 我们的方法能够实现良好的预测精度。这一普遍观察与由方程 4 的均方误差损失所引起的优化景观密切相关, 并定义了一个开放的问题

图1总结了我们的数据驱动方法求解Burgers方程的结果。具体来说, 给定一组 $N_u = 100$ 个随机分布的初始和边界数据, 我们通过使用公式(4)的均方误差损失训练一个9层深度神经网络的所有3021个参数来学习潜在解 $u(t, x)$ 。每个隐藏层包含20个神经元, 并使用双曲正切激活函数。通常, 神经网络应具备足够的逼近能力以满足预期需求

complexity of $u(t, x)$. However, in this example, our choice aims to highlight the robustness of the proposed method with respect to the well known issue of over-fitting. Specifically, the term in MSE_f in equation (4) acts as a regularization mechanism that penalizes solutions that do not satisfy equation (3). Therefore, a key property of *physics informed neural networks* is that they can be effectively trained using small data sets; a setting often encountered in the study of physical systems for which the cost of data acquisition may be prohibitive.

The top panel of Figure 1 shows the predicted spatio-temporal solution $u(t, x)$, along with the locations of the initial and boundary training data. We must underline that, unlike any classical numerical method for solving partial differential equations, this prediction is obtained without any sort of discretization of the spatio-temporal domain. The exact solution for this problem is analytically available [14], and the resulting prediction error is measured at $6.7 \cdot 10^{-4}$ in the relative $\mathcal{L}_2$ -norm. Note that this error is about two orders of magnitude lower than the one reported in our previous work on data-driven solution of partial differential equation using Gaussian processes [9]. A more detailed assessment of the predicted solution is presented in the bottom panel of figure 1. In particular, we present a comparison between the exact and the predicted solutions at different time instants $t = 0.25, 0.50, 0.75$. Using only a handful of initial and boundary data, the *physics informed neural network* can accurately capture the intricate non-linear behavior of the Burgers' equation that leads to the development of a sharp internal layer around $t = 0.4$. The latter is notoriously hard to accurately resolve with classical numerical methods and requires a laborious spatio-temporal discretization of equation (3).

To further analyze the performance of our method, we have performed the following systematic studies to quantify its predictive accuracy for different number of training and collocation points, as well as for different neural network architectures. In table 1 we report the resulting relative $\mathcal{L}_2$ error for different number of initial and boundary training data N_u and different number of collocation points N_f , while keeping the 9-layer network architecture fixed. The general trend shows increased prediction accuracy as the total number of training data N_u is increased, given a sufficient number of collocation points N_f . This observation highlights a key strength of *physics informed neural networks*: by encoding the structure of the underlying phys-

$u(t, x)$ 的复杂性。然而，在这个例子中，我们的选择旨在强调所提出方法在众所周知的过拟合问题上的稳健性。具体来说，方程(4)中 MSE_f 项作为正则化机制，惩罚不满足方程(3)的解。因此，*physics informed neural networks*的一个关键特性是它们可以使用小数据集进行有效训练；这种情况在研究物理系统时经常遇到，因为数据获取的成本可能非常高。

图1的顶部面板显示了预测的时空解 $u(t, x)$ ，以及初始和边界训练数据的位置。我们必须强调，与任何经典的求解偏微分方程的数值方法不同，这一预测是在没有对时空域进行任何形式的离散化的情况下得到的。该问题的精确解在解析上是可得的 [14]，并且所得的预测误差在相对 $\mathcal{L}_2$ 范数下为 $6.7 \cdot 10^{-4}$ 。注意，该误差比我们之前使用高斯过程对偏微分方程进行数据驱动求解的工作中报告的误差低大约两个数量级 [9]。对预测解的更详细评估呈现在图1的底部面板中。特别地，我们展示了在不同时间点 $t = 0.25, 0.50, 0.75$ 精确解与预测解的比较。仅使用少量初始和边界数据

为了进一步分析我们方法的性能，我们进行了以下系统研究，以量化其不同训练点和配点数量以及不同神经网络架构下的预测精度。在表1中，我们报告了在保证9层网络架构不变的情况下，不同初始和边界训练数据数量以及不同配点数量的相对 $\mathcal{L}_2$ 误差。总体趋势显示，在配点数量充足的情况下，随着总训练数据数量的增加，预测精度提高。这一观察结果突出了 *physics informed neural networks* 的一个关键优势：通过编码底层物理结构

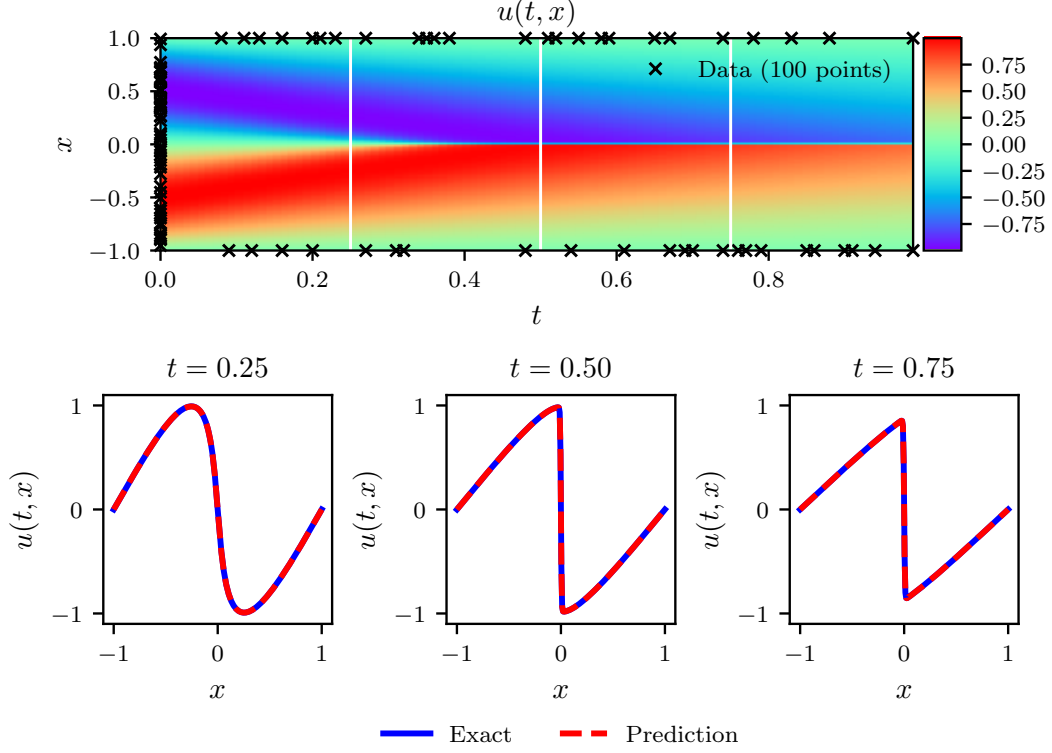


Figure 1: *Burgers' equation*: *Top*: Predicted solution $u(t, x)$ along with the initial and boundary training data. In addition we are using 10,000 collocation points generated using a Latin Hypercube Sampling strategy. *Bottom*: Comparison of the predicted and exact solutions corresponding to the three temporal snapshots depicted by the white vertical lines in the top panel. The relative $\mathcal{L}_2$ error for this case is $6.7 \cdot 10^{-4}$. Model training took approximately 60 seconds on a single NVIDIA Titan X GPU card.

ical law through the collocation points N_f , one can obtain a more accurate and data-efficient learning algorithm.¹ Finally, table 2 shows the resulting relative $\mathcal{L}_2$ for different number of hidden layers, and different number of neurons per layer, while the total number of training and collocation points is kept fixed to $N_u = 100$ and $N_f = 10,000$, respectively. As expected, we observe that as the number of layers and neurons is increased (hence the capacity of the neural network to approximate more complex functions), the

¹Note that the case $N_f = 0$ corresponds to a standard neural network model, i.e., a neural network that does not take into account the underlying governing equation.

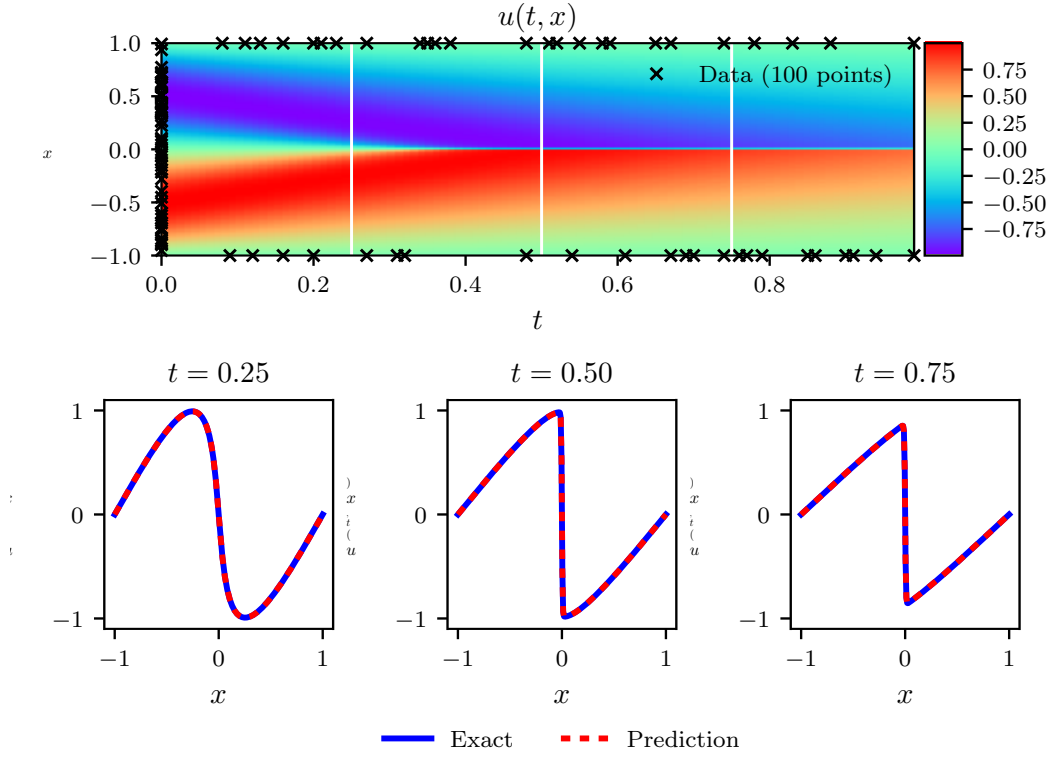


图 1: *Burgers' equation*: *Top*: 预测解 $u(t, x)$ 以及初始和边界训练数据。此外，我们使用拉丁超立方抽样策略生成了 10,000 个配点。 *Bottom*: 对应顶面中白色垂直线描绘的三个时间快照，预测解和精确解的比较。该情况下的相对 $\mathcal{L}_2$ 误差为 $6.7 \cdot 10^{-4}$ 。模型训练在单个 NVIDIA Titan X GPU 上大约耗时 60 秒。

通过配点 N_f 可以获得更准确且数据效率更高的学习算法。¹ 最后，表 2 显示了在隐藏层数量和每层神经元数量不同情况下的相对结果 $\mathcal{L}_2$ ，同时训练点和配点的总数保持固定，分别为 $N_u = 100$ 和 $N_f = 10,000$ 。正如预期的那样，我们观察到，随着层数和神经元数量的增加（因此神经网络近似更复杂函数的能力增强），

¹Note that the case $N_f = 0$ corresponds to a standard neural network model, i.e., a neural network that does not take into account the underlying governing equation.

$N_u \backslash N_f$	2000	4000	6000	7000	8000	10000
20	2.9e-01	4.4e-01	8.9e-01	1.2e+00	9.9e-02	4.2e-02
40	6.5e-02	1.1e-02	5.0e-01	9.6e-03	4.6e-01	7.5e-02
60	3.6e-01	1.2e-02	1.7e-01	5.9e-03	1.9e-03	8.2e-03
80	5.5e-03	1.0e-03	3.2e-03	7.8e-03	4.9e-02	4.5e-03
100	6.6e-02	2.7e-01	7.2e-03	6.8e-04	2.2e-03	6.7e-04
200	1.5e-01	2.3e-03	8.2e-04	8.9e-04	6.1e-04	4.9e-04

Table 1: *Burgers' equation*: Relative $\mathcal{L}_2$ error between the predicted and the exact solution $u(t, x)$ for different number of initial and boundary training data N_u , and different number of collocation points N_f . Here, the network architecture is fixed to 9 layers with 20 neurons per hidden layer.

$\text{Layers} \backslash \text{Neurons}$	10	20	40
2	7.4e-02	5.3e-02	1.0e-01
4	3.0e-03	9.4e-04	6.4e-04
6	9.6e-03	1.3e-03	6.1e-04
8	2.5e-03	9.6e-04	5.6e-04

Table 2: *Burgers' equation*: Relative $\mathcal{L}_2$ error between the predicted and the exact solution $u(t, x)$ for different number of hidden layers and different number of neurons per layer. Here, the total number of training and collocation points is fixed to $N_u = 100$ and $N_f = 10,000$, respectively.

predictive accuracy is increased.

2.2. Example (Schrödinger Equation)

This example aims to highlight the ability of our method to handle periodic boundary conditions, complex-valued solutions, as well as different types of nonlinearities in the governing partial differential equations. The one-dimensional nonlinear Schrödinger equation is a classical field equation that is used to study quantum mechanical systems, including nonlinear wave propagation in optical fibers and/or waveguides, Bose-Einstein condensates, and plasma waves. In optics, the nonlinear term arises from the intensity dependent index of refraction of a given material. Similarly, the nonlinear term for Bose-Einstein condensates is a result of the mean-field interactions of an interacting, N-body system. The nonlinear Schrödinger equation along

$N_u \backslash N_f$	2000	4000	6000	7000	8000	10000
20	2.9e-01	4.4e-01	8.9e-01	1.2e+00	9.9e-02	4.2e-02
40	6.5e-02	1.1e-02	5.0e-01	9.6e-03	4.6e-01	7.5e-02
60	3.6e-01	1.2e-02	1.7e-01	5.9e-03	1.9e-03	8.2e-03
80	5.5e-03	1.0e-03	3.2e-03	7.8e-03	4.9e-02	4.5e-03
100	6.6e-02	2.7e-01	7.2e-03	6.8e-04	2.2e-03	6.7e-04
200	1.5e-01	2.3e-03	8.2e-04	8.9e-04	6.1e-04	4.9e-04

表 1: *Burgers' equation*: 在不同数量的初始和边界训练数据 N_u 以及不同数量的配点 N_f 下, 预测解与精确解 $u(t, x)$ 之间的相对 $\mathcal{L}_2$ 误差 *Burgers' equation*:。这里, 网络结构固定为 9 层, 每个隐藏层包含 20 个神经元。

$\text{Layers} \backslash \text{Neurons}$	10	20	40
2	7.4e-02	5.3e-02	1.0e-01
4	3.0e-03	9.4e-04	6.4e-04
6	9.6e-03	1.3e-03	6.1e-04
8	2.5e-03	9.6e-04	5.6e-04

表 2: *Burgers' equation*: 预测解与精确解之间的相对 $\mathcal{L}_2$ 误差 $u(t, x)$, 针对不同隐藏层数和每层不同神经元数量。这里, 训练点和匹配点的总数分别固定为 $N_u = 100$ 和 $N_f = 10,000$ 。

预测精度提高了。

2.2. Example (Shrödinger Equation)

这个例子旨在突出我们方法处理周期性边界条件、复值解以及控制偏微分方程中不同类型非线性的能力。一维非线性薛定谔方程是一种经典的场方程, 用于研究量子力学系统, 包括光纤和/或波导中的非线性波传播、玻色-爱因斯坦凝聚体以及等离子体波。在光学中, 非线性项来源于给定材料的折射率随光强度变化的特性。类似地, 玻色-爱因斯坦凝聚体的非线性项是由相互作用的 N 体系统的平均场相互作用引起的。非线性薛定谔方程沿

with periodic boundary conditions is given by

$$\begin{aligned} ih_t + 0.5h_{xx} + |h|^2h &= 0, \quad x \in [-5, 5], \quad t \in [0, \pi/2], \\ h(0, x) &= 2 \operatorname{sech}(x), \\ h(t, -5) &= h(t, 5), \\ h_x(t, -5) &= h_x(t, 5), \end{aligned} \quad (5)$$

where $h(t, x)$ is the complex-valued solution. Let us define $f(t, x)$ to be given by

$$f := ih_t + 0.5h_{xx} + |h|^2h,$$

and proceed by placing a complex-valued neural network prior on $h(t, x)$. In fact, if u denotes the real part of h and v is the imaginary part, we are placing a multi-out neural network prior on $h(t, x) = [u(t, x) \ v(t, x)]$. This will result in the complex-valued (multi-output) *physic informed neural network* $f(t, x)$. The shared parameters of the neural networks $h(t, x)$ and $f(t, x)$ can be learned by minimizing the mean squared error loss

$$MSE = MSE_0 + MSE_b + MSE_f, \quad (6)$$

where

$$\begin{aligned} MSE_0 &= \frac{1}{N_0} \sum_{i=1}^{N_0} |h(0, x_0^i) - h_0^i|^2, \\ MSE_b &= \frac{1}{N_b} \sum_{i=1}^{N_b} (|h^i(t_b^i, -5) - h^i(t_b^i, 5)|^2 + |h_x^i(t_b^i, -5) - h_x^i(t_b^i, 5)|^2), \end{aligned}$$

and

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$

Here, $\{x_0^i, h_0^i\}_{i=1}^{N_0}$ denotes the initial data, $\{t_b^i\}_{i=1}^{N_b}$ corresponds to the collocation points on the boundary, and $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ represents the collocation points on $f(t, x)$. Consequently, MSE_0 corresponds to the loss on the initial data, MSE_b enforces the periodic boundary conditions, and MSE_f penalizes the Schrödinger equation not being satisfied on the collocation points.

In order to assess the accuracy of our method, we have simulated equation

在周期性边界条件下给出

$$\begin{aligned} ih_t + 0.5h_{xx} + |h|^2h &= 0, \quad x \in [-5, 5], \quad t \in [0, \pi/2], \\ h(0, x) &= 2 \operatorname{sech}(x), \\ h(t, -5) &= h(t, 5), \\ h_x(t, -5) &= h_x(t, 5), \end{aligned} \quad (5)$$

其中 $h(t, x)$ 是复值解。让我们定义 $f(t, x)$ 如下

$$f := ih_t + 0.5h_{xx} + |h|^2h,$$

并通过在 $h(t, x)$ 上放置一个复值神经网络先验来继续。事实上，如果 u 表示 h 的实部，而 v 是虚部，我们是在 $h(t, x) = [u(t, x) \ v(t, x)]$ 上放置一个多输出神经网络先验。这将导致复值（多输出）*physic informed neural network* $f(t, x)$ 。神经网络 $h(t, x)$ 和 $f(t, x)$ 的共享参数可以通过最小化均方误差损失来学习

$$MSE = MSE_0 + MSE_b + MSE_f, \quad (6)$$

哪里

$$MSE_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |h(0, x_0^i) - h_0^i|^2,$$

$$MSE_b = \frac{1}{N_b} \sum_{i=1}^{N_b} (|h^i(t_b^i, -5) - h^i(t_b^i, 5)|^2 + |h_x^i(t_b^i, -5) - h_x^i(t_b^i, 5)|^2),$$

和

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$

这里， $\{x_0^i, h_0^i\}_{i=1}^{N_0}$ 表示初始数据， $\{t_b^i\}_{i=1}^{N_b}$ 对应边界上的配点，而 $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ 表示 $f(t, x)$ 上的配点。因此， MSE_0 对应初始数据的损失， MSE_b 强制执行周期性边界条件， MSE_f 对在配点上不满足谢定方程进行惩罚。

为了评估我们方法的准确性，我们已经对方程进行了模拟

(5) using conventional spectral methods to create a high-resolution data set. Specifically, starting from an initial state $h(0, x) = 2 \operatorname{sech}(x)$ and assuming periodic boundary conditions $h(t, -5) = h(t, 5)$ and $h_x(t, -5) = h_x(t, 5)$, we have integrated equation (5) up to a final time $t = \pi/2$ using the Chebfun package [22] with a spectral Fourier discretization with 256 modes and a fourth-order explicit Runge-Kutta temporal integrator with time-step $\Delta t = \pi/2 \cdot 10^{-6}$. Under our data-driven setting, all we observe are measurements $\{x_0^i, h_0^i\}_{i=1}^{N_0}$ of the latent function $h(t, x)$ at time $t = 0$. In particular, the training set consists of a total of $N_0 = 50$ data points on $h(0, x)$ randomly parsed from the full high-resolution data-set, as well as $N_b = 50$ randomly sampled collocation points $\{t_b^i\}_{i=1}^{N_b}$ for enforcing the periodic boundaries. Moreover, we have assumed $N_f = 20,000$ randomly sampled collocation points used to enforce equation (5) inside the solution domain. All randomly sampled point locations were generated using a space filling Latin Hypercube Sampling strategy [23].

Here our goal is to infer the entire spatio-temporal solution $h(t, x)$ of the Schrödinger equation (5). We chose to jointly represent the latent function $h(t, x) = [u(t, x) \ v(t, x)]$ using a 5-layer deep neural network with 100 neurons per layer and a hyperbolic tangent activation function. Figure 2 summarizes the results of our experiment. Specifically, the top panel of figure 2 shows the magnitude of the predicted spatio-temporal solution $|h(t, x)| = \sqrt{u^2(t, x) + v^2(t, x)}$, along with the locations of the initial and boundary training data. The resulting prediction error is validated against the test data for this problem, and is measured at $1.97 \cdot 10^{-3}$ in the relative $\mathcal{L}_2$ -norm. A more detailed assessment of the predicted solution is presented in the bottom panel of Figure 2. In particular, we present a comparison between the exact and the predicted solutions at different time instants $t = 0.59, 0.79, 0.98$. Using only a handful of initial data, the *physics informed neural network* can accurately capture the intricate nonlinear behavior of the Schrödinger equation.

One potential limitation of the continuous time neural network models considered so far, stems from the need to use a large number of collocation points N_f in order to enforce physics informed constraints in the entire spatio-temporal domain. Although this poses no significant issues for problems in one or two spatial dimensions, it may introduce a severe bottleneck in higher dimensional problems, as the total number of collocation points needed

(5) 使用传统光谱方法创建高分辨率数据集。具体来说，从初始状态 $h(0, x) = 2 \operatorname{sech}(x)$ 出发，并假设周期边界条件 $h(t, -5) = h(t, 5)$ 和 $h_x(t, -5) = h_x(t, 5)$ ，我们使用 Chebfun 软件包 [22] 对方程 (5) 积分直到最终时间 $t = \pi/2$ ，通过具有 256 模的光谱傅里叶离散化和时间步长为 $\Delta t = \pi/2 \cdot 10^{-6}$ 的四阶显式 Runge-Kutta 时间积分器。在我们的数据驱动设置下，我们只观察在时间 $t = 0$ 时潜在函数 $h(t, x)$ 的测量值 $\{x_0^i, h_0^i\}_{i=1}^{N_0}$ 。具体来说，训练集总共包括 $N_0 = 50$ 个数据点在 $h(0, x)$ 随机抽取自完整高分辨率数据集，以及 $N_b = 50$ 个随机采样的重碰点 $\{t_b^i\}_{i=1}^{N_b}$ 用于强制周期边界。此外，我们假设 $N_f = 20,000$ 个随机采样的重碰点用于在求解域内部强制方程 (5)。所有随机采样点位

在这里，我们的目标是推断薛定谔方程(5)的整个时空解 $h(t, x)$ 。我们选择使用一个 5 层深度神经网络，每层 100 个神经元，并使用双曲正切激活函数，共同表示潜在函数 $h(t, x) = [u(t, x) \ v(t, x)]$ 。图 2 总结了我们的实验结果。具体而言，图 2 的顶部面板显示了预测的时空解 $|h(t, x)| = \sqrt{u^2(t, x) + v^2(t, x)}$ 的幅值，以及初始和边界训练数据的位置。所得预测误差已针对该问题的测试数据进行验证，并在相对 $\mathcal{L}_2$ -范数下测量为 $1.97 \cdot 10^{-3}$ 。图 2 底部面板提供了对预测解的更详细评估。特别地，我们展示了在不同时间点 $t = 0.59, 0.79, 0.98$ 的精确解与预测解的比较。仅使用少量初始数据，*physics informed neural network* 就能够准确捕捉到

到目前为止所考虑的连续时间神经网络模型的一个潜在局限性，源于需要使用大量的配点 N_f 以便在整个时空域中强制执行物理信息约束。虽然对于一维或二维空间问题这并不会带来显著问题，但在更高维问题中，这可能会引入严重的瓶颈，因为所需的配点总数

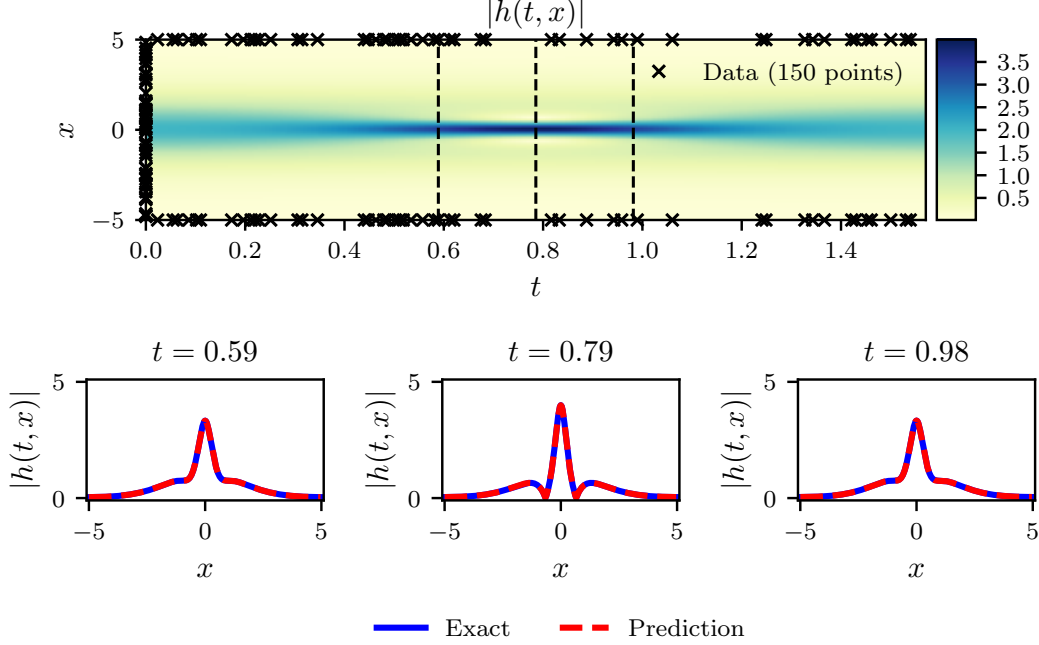


Figure 2: *Schrödinger equation*: *Top*: Predicted solution $|h(t, x)|$ along with the initial and boundary training data. In addition we are using 20,000 collocation points generated using a Latin Hypercube Sampling strategy. *Bottom*: Comparison of the predicted and exact solutions corresponding to the three temporal snapshots depicted by the dashed vertical lines in the top panel. The relative $\mathcal{L}_2$ error for this case is $1.97 \cdot 10^{-3}$.

to globally enforce a physics informed constrain (i.e., in our case a partial differential equation) will increase exponentially. In the next section, we put forth a different approach that circumvents the need for collocation points by introducing a more structured neural network representation leveraging the classical Runge-Kutta time-stepping schemes [24].

3. Discrete Time Models

Let us apply the general form of Runge-Kutta methods with q stages [24] to equation (1) and obtain

$$\begin{aligned} u^{n+c_i} &= u^n - \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q, \\ u^{n+1} &= u^n - \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}]. \end{aligned} \quad (7)$$

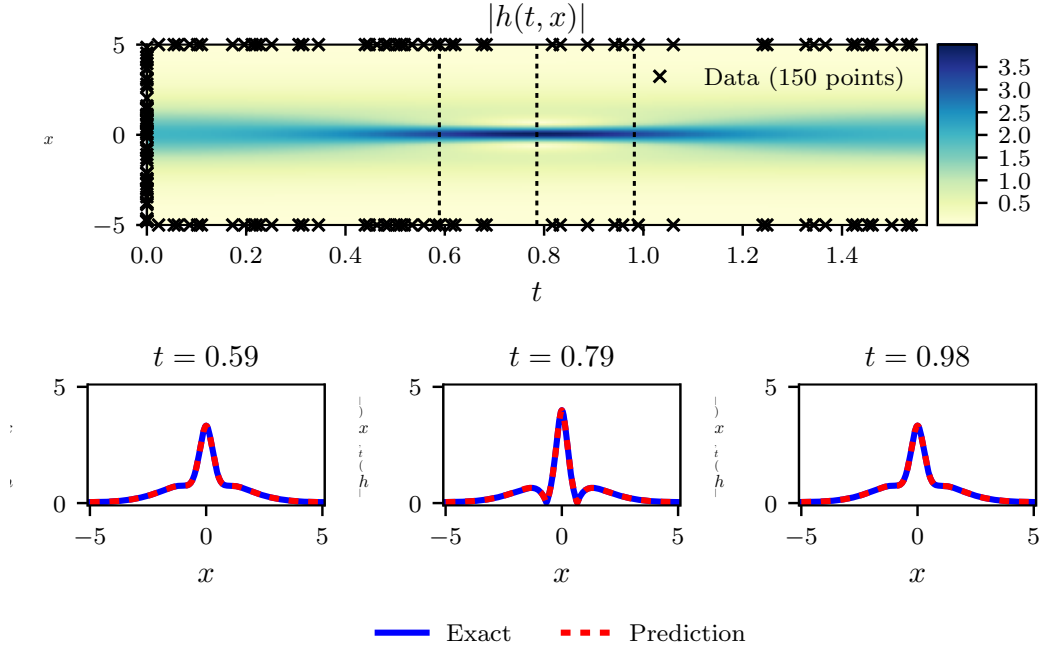


图2: *Shrödinger equation*: Top: 预测解 $|h(t, x)|$ 以及初始和边界训练数据。此外，我们使用拉丁超立方抽样策略生成了20,000个配置点。Bottom: 对应顶部面板中虚线所示的三个时间快照的预测解和精确解的比较。该情况下的相对 $\mathcal{L}_2$ 误差为 $1.97 \cdot 10^{-3}$ 。

全局施加物理信息约束（即在我们的例子中是一个偏微分方程）的计算量将呈指数增加。在下一节中，我们提出了一种不同的方法，通过引入更结构化的神经网络表示并利用经典的龙格-库塔时间步进方案[24]，从而避免了对配点的需求。

3. Discrete Time Models

让我们将具有 q 阶段的龙格-库塔方法的一般形式 [24] 应用于方程 (1) 并得到

$$\begin{aligned} u^{n+c_i} &= u^n - \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q, \\ u^{n+1} &= u^n - \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}]. \end{aligned} \quad (7)$$

Here, $u^{n+c_j}(x) = u(t^n + c_j\Delta t, x)$ for $j = 1, \dots, q$. This general form encapsulates both implicit and explicit time-stepping schemes, depending on the choice of the parameters $\{a_{ij}, b_j, c_j\}$. Equations (7) can be equivalently expressed as

$$\begin{aligned} u^n &= u_i^n, \quad i = 1, \dots, q, \\ u^n &= u_{q+1}^n, \end{aligned} \tag{8}$$

where

$$\begin{aligned} u_i^n &:= u^{n+c_i} + \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q, \\ u_{q+1}^n &:= u^{n+1} + \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}]. \end{aligned} \tag{9}$$

We proceed by placing a multi-output neural network prior on

$$[u^{n+c_1}(x), \dots, u^{n+c_q}(x), u^{n+1}(x)]. \tag{10}$$

This prior assumption along with equations (9) result in a *physics informed neural network* that takes x as an input and outputs

$$[u_1^n(x), \dots, u_q^n(x), u_{q+1}^n(x)]. \tag{11}$$

3.1. Example (Burgers' Equation)

To highlight the key features of the discrete time representation we revisit the problem of data-driven solution of the Burgers' equation. For this case, the nonlinear operator in equation (9) is given by

$$\mathcal{N}[u^{n+c_j}] = u^{n+c_j} u_x^{n+c_j} - (0.01/\pi) u_{xx}^{n+c_j},$$

and the shared parameters of the neural networks (10) and (11) can be learned by minimizing the sum of squared errors

$$SSE = SSE_n + SSE_b, \tag{12}$$

where

$$SSE_n = \sum_{j=1}^{q+1} \sum_{i=1}^{N_n} |u_j^n(x^{n,i}) - u^{n,i}|^2,$$

在这里, $u^{n+c_j}(x) = u(t^n + c_j\Delta t, x)$ 用于 $j = 1, \dots, q$ 。这个一般形式包含了隐式和显式时间步进方法, 取决于参数 $\{a_{ij}, b_j, c_j\}$ 的选择。方程 (7) 可以等价地表示为

$$\begin{aligned} u^n &= u_i^n, \quad i = 1, \dots, q, \\ u^n &= u_{q+1}^n, \end{aligned} \tag{8}$$

哪里

$$\begin{aligned} u_i^n &:= u^{n+c_i} + \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q, \\ u_{q+1}^n &:= u^{n+1} + \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}]. \end{aligned} \tag{9}$$

我们通过在……上放置多输出神经网络先验来进行

$$[u^{n+c_1}(x), \dots, u^{n+c_q}(x), u^{n+1}(x)]. \tag{10}$$

这个先验假设以及方程 (9) 导致了一个 *physics informed neural network*, 它以 x 作为输入并产生输出

$$[u_1^n(x), \dots, u_q^n(x), u_{q+1}^n(x)]. \tag{11}$$

3.1. Example (Burgers' Equation)

为了突出离散时间表示的关键特征, 我们重新审视了数据驱动的 Burgers 方程求解问题。在这种情况下, 方程 (9) 中的非线性算子由下式给出

$$\mathcal{N}[u^{n+c_j}] = u^{n+c_j} u_x^{n+c_j} - (0.01/\pi) u_{xx}^{n+c_j},$$

神经网络 (10) 和 (11) 的共享参数可以通过最小化平方误差和来学习

$$SSE = SSE_n + SSE_b, \tag{12}$$

哪里

$$SSE_n = \sum_{j=1}^{q+1} \sum_{i=1}^{N_n} |u_j^n(x^{n,i}) - u^{n,i}|^2,$$

and

$$SSE_b = \sum_{i=1}^q (|u^{n+c_i}(-1)|^2 + |u^{n+c_i}(1)|^2) + |u^{n+1}(-1)|^2 + |u^{n+1}(1)|^2.$$

Here, $\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ corresponds to the data at time t^n . The Runge-Kutta scheme now allows us to infer the latent solution $u(t, x)$ in a sequential fashion. Starting from initial data $\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ at time t^n and data at the domain boundaries $x = -1$ and $x = 1$, we can use the aforementioned loss function (12) to train the networks of (10), (11), and predict the solution at time t^{n+1} . A Runge-Kutta time-stepping scheme would then use this prediction as initial data for the next step and proceed to train again and predict $u(t^{n+2}, x)$, $u(t^{n+3}, x)$, etc., one step at a time.

In classical numerical analysis, these steps are usually confined to be small due to stability constraints for explicit schemes or computational complexity constrains for implicit formulations [24]. These constraints become more severe as the total number of Runge-Kutta stages q is increased, and, for most problems of practical interest, one needs to take thousands to millions of such steps until the solution is resolved up to a desired final time. In sharp contrast to classical methods, here we can employ implicit Runge-Kutta schemes with an arbitrarily large number of stages at effectively no extra cost.² This enables us to take very large time steps while retaining stability and high predictive accuracy, therefore allowing us to resolve the entire spatio-temporal solution in a single step.

The result of applying this process to the Burgers' equation is presented in figure 3. For illustration purposes, we start with a set of $N_n = 250$ initial data at $t = 0.1$, and employ a *physics informed neural network* induced by an implicit Runge-Kutta scheme with 500 stages to predict the solution at time $t = 0.9$ in a single step. The theoretical error estimates for this scheme predict a temporal error accumulation of $\mathcal{O}(\Delta t^{2q})$ [24], which in our case translates into an error way below machine precision, i.e., $\Delta t^{2q} = 0.8^{1000} \approx 10^{-97}$. To our knowledge, this is the first time that an implicit Runge-Kutta scheme

²To be precise, it is only the number of parameters in the last layer of the neural network that increases linearly with the total number of stages.

和

$$SSE_b = \sum_{i=1}^q (|u^{n+c_i}(-1)|^2 + |u^{n+c_i}(1)|^2) + |u^{n+1}(-1)|^2 + |u^{n+1}(1)|^2.$$

这里, $\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ 对应于时间 t^n 的数据。Runge-Kutta 方案现在允许我们以顺序的方式推断潜在解 $u(t, x)$ 。从时间 t^n 的初始数据 $\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ 以及域边界的数据 $x = -1$ 和 $x = 1$ 开始, 我们可以使用上述损失函数 (12) 来训练 (10)、(11) 的网络, 并预测时间 t^{n+1} 的解。然后, Runge-Kutta 时间步进方案将使用该预测作为下一步的初始数据, 并继续训练, 然后预测 $u(t^{n+2}, x)$ 、 $u(t^{n+3}, x)$ 等, 一次一步地进行。

在经典数值分析中, 由于显式方法的稳定性约束或隐式公式的计算复杂性约束, 这些步骤通常被限制得很小 [24]。当 Runge-Kutta 阶数 q 增加时, 这些约束会变得更加严格, 对于大多数实际问题, 需要进行数千到数百万步才能将解计算到所需的最终时间。与经典方法形成鲜明对比的是, 在这里我们可以以几乎无额外成本的方式使用任意大量阶段的隐式 Runge-Kutta 方案²。这使我们能够在保持稳定性和高预测精度的同时采取非常大的时间步长, 从而允许我们在一步中解析整个时空解。

将此过程应用于 Burgers 方程的结果如图 3 所示。为了说明起见, 我们从一组在 0.1 时的 250 个初始数据开始, 并使用由具有 500 个阶段的隐式 Runge-Kutta 方案引起的诱导方案, 在单步中预测时间 0.9 的解。该方案的理论误差估计预测了一个时间误差累积 [24], 在我们的情况下, 这转化为远低于机器精度的误差, 即 0.8×10^{-23} 。据我们所知, 这是隐式 Runge-Kutta 方案的首次实现

²To be precise, it is only the number of parameters in the last layer of the neural network that increases linearly with the total number of stages.

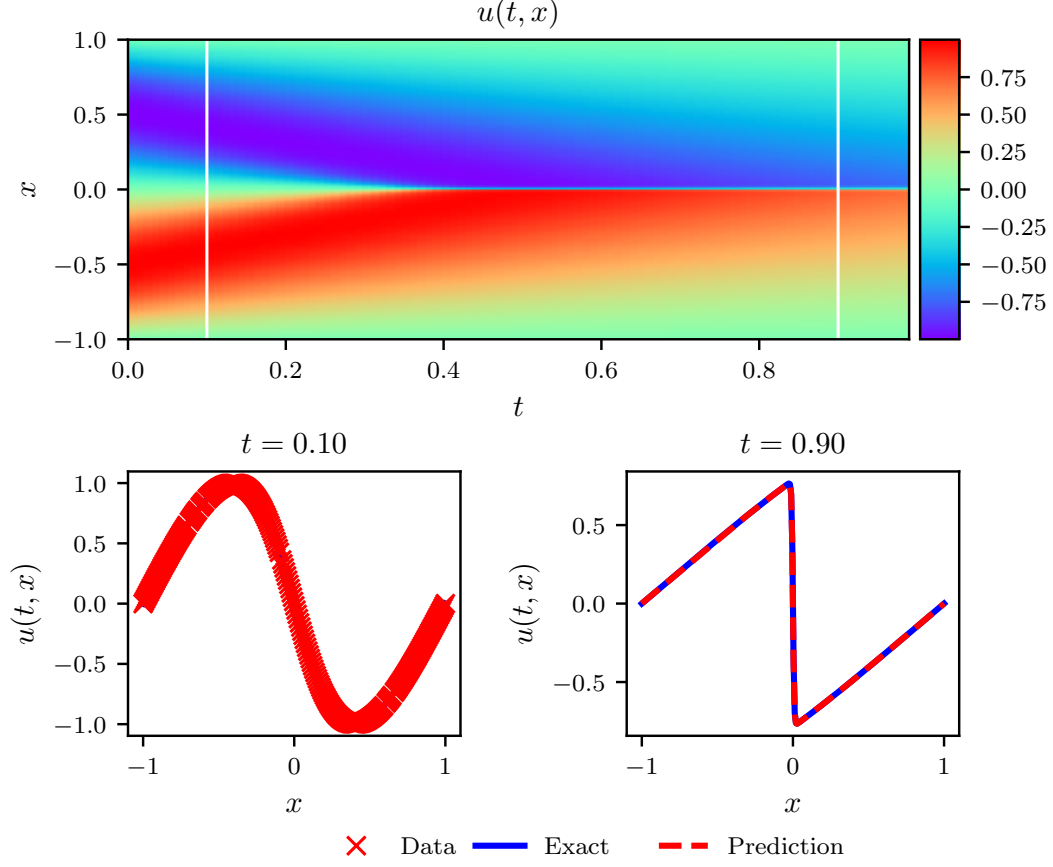


Figure 3: *Burgers equation*: Top: Solution $u(t, x)$ along with the location of the initial training snapshot at $t = 0.1$ and the final prediction snapshot at $t = 0.9$. Bottom: Initial training data and final prediction at the snapshots depicted by the white vertical lines in the top panel. The relative $\mathcal{L}_2$ error for this case is $8.2 \cdot 10^{-4}$.

of that high-order has ever been used. Remarkably, starting from smooth initial data at $t = 0.1$ we can predict the nearly discontinuous solution at $t = 0.9$ in a single time-step with a relative $\mathcal{L}_2$ error of $8.2 \cdot 10^{-4}$. This error is two orders of magnitude lower than the one reported in [9], and it is entirely attributed to the neural network’s capacity to approximate $u(t, x)$, as well as to the degree that the sum of squared errors loss allows interpolation of the training data. The network architecture used here consists of 4 layers with 50 neurons in each hidden layer.

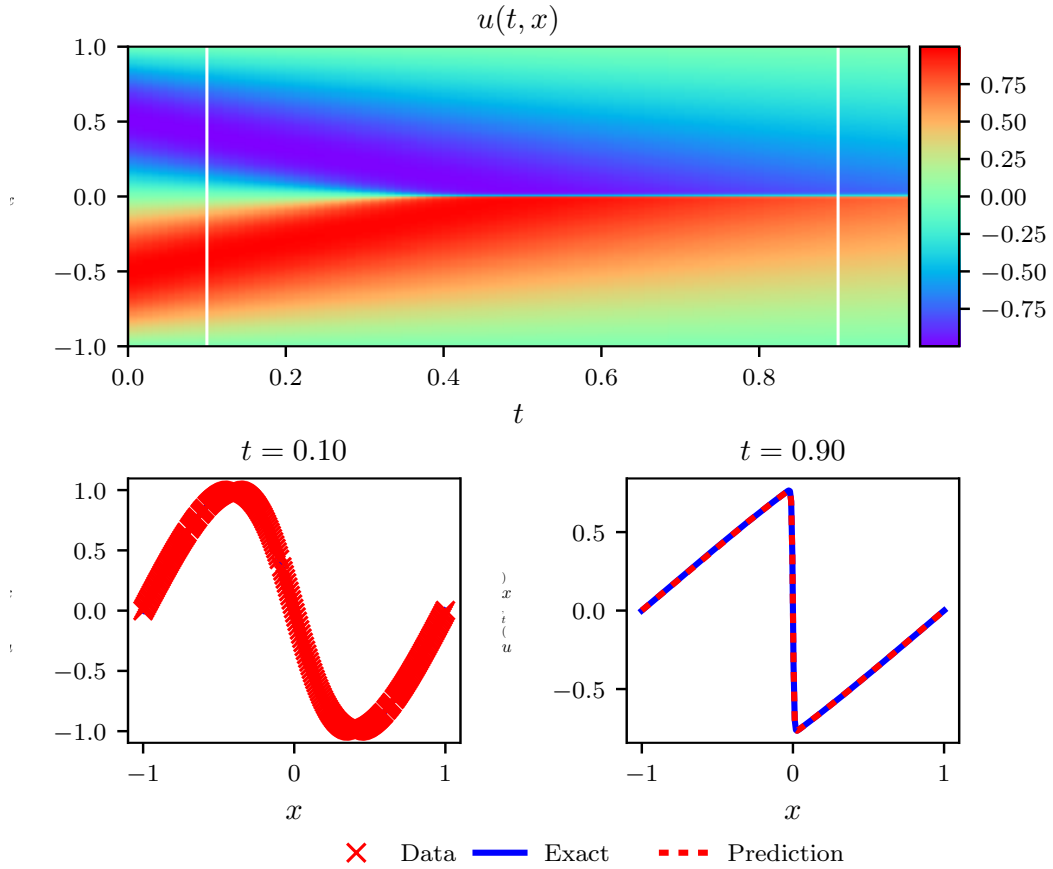


图 3: *Burgers equation*: Top: 解决方案 $u(t, x)$ 以及初始训练快照位于 $t = 0.1$ 和最终预测快照位于 $t = 0.9$ 的位置。Bottom: 顶部面板中白色垂直线所示的快照对应的初始训练数据和最终预测。该情况下的相对 $\mathcal{L}_2$ 误差为 $8.2 \cdot 10^{-4}$ 。

迄今为止从未使用过如此高阶的方法。值得注意的是，从平滑初始数据 $t = 0.1$ 开始，我们可以在单个时间步内预测 $t = 0.9$ 的几乎不连续解，相对 $\mathcal{L}_2$ 误差为 $8.2 \cdot 10^{-4}$ 。这个误差比文献 [9] 报告的低两个数量级，这完全归因于神经网络近似 $u(t, x)$ 的能力，以及平方误差总和损失允许对训练数据进行插值的程度。这里使用的网络架构由 4 层组成，每个隐藏层有 50 个神经元。

Layers \ Neurons	Neurons		
	10	25	50
1	4.1e-02	4.1e-02	1.5e-01
2	2.7e-03	5.0e-03	2.4e-03
3	3.6e-03	1.9e-03	9.5e-04

Table 3: *Burgers' equation*: Relative final prediction error measure in the $\mathcal{L}_2$ norm for different number of hidden layers and neurons in each layer. Here, the number of Runge-Kutta stages is fixed to 500 and the time-step size to $\Delta t = 0.8$.

A detailed systematic study to quantify the effect of different network architectures is presented in table 3. By keeping the number of Runge-Kutta stages fixed to $q = 500$ and the time-step size to $\Delta t = 0.8$, we have varied the number of hidden layers and the number of neurons per layer, and monitored the resulting relative $\mathcal{L}_2$ error for the predicted solution at time $t = 0.9$. Evidently, as the neural network capacity is increased the predictive accuracy is enhanced.

The key parameters controlling the performance of our discrete time algorithm are the total number of Runge-Kutta stages q and the time-step size Δt . In table 4 we summarize the results of an extensive systematic study where we fix the network architecture to 4 hidden layers with 50 neurons per layer, and vary the number of Runge-Kutta stages q and the time-step size Δt . Specifically, we see how cases with low numbers of stages fail to yield accurate results when the time-step size is large. For instance, the case $q = 1$ corresponding to the classical trapezoidal rule, and the case $q = 2$ corresponding to the 4th-order Gauss-Legendre method, cannot retain their predictive accuracy for time-steps larger than 0.2, thus mandating a solution strategy with multiple time-steps of small size. On the other hand, the ability to push the number of Runge-Kutta stages to 32 and even higher allows us to take very large time steps, and effectively resolve the solution in a single step without sacrificing the accuracy of our predictions. Moreover, numerical stability is not sacrificed either as implicit Runge-Kutta is the only family of time-stepping schemes that remain A-stable regardless of their order, thus making them ideal for stiff problems [24]. These properties are unprecedented for an algorithm of such implementation simplicity, and illustrate one of the key highlights of our discrete time approach.

Layers \ Neurons	Neurons		
	10	25	50
1	4.1e-02	4.1e-02	1.5e-01
2	2.7e-03	5.0e-03	2.4e-03
3	3.6e-03	1.9e-03	9.5e-04

表 3: 在 $\mathcal{L}_2$ 范数下, 不同隐藏层数量及每层神经元数量的 *Burgers' equation*: 相对最终预测误差度量。这里, Runge-Kutta 阶数固定为 500, 时间步长固定为 $\Delta t = 0.8$ 。

在表3中展示了一项详细的系统研究, 用于量化不同网络架构的影响。通过将Runge-Kutta阶段数固定为500, 时间步长固定为0.08, 我们改变了隐藏层的数量和每层的神经元数量, 并监测在时间0.09预测解的相对误差。显然, 随着神经网络容量的增加, 预测精度得到了提高。

控制我们离散时间算法性能的关键参数是Runge-Kutta阶段的总数 q 和时间步长 Δt 。在表4中, 我们总结了一项广泛系统研究的结果, 在该研究中, 我们将网络架构固定为4个隐藏层, 每层50个神经元, 并改变Runge-Kutta阶段的数量 q 和时间步长 Δt 。具体来说, 我们观察到, 当时间步长较大时, 阶段数量较少的情况无法产生准确结果。例如, 案例 $q = 1$ 对应经典梯形法, 以及案例 $q = 2$ 对应4th 阶高斯-勒让德方法, 在时间步长大于0.2时无法保持其预测精度, 因此需要采用多小步长的解法策略。另一方面, 将Runge-Kutta阶段数提升到32甚至更高, 使我们能够采用非常大的时间步长, 并有效地在一次计算中解析解。

$q \backslash \Delta t$	0.2	0.4	0.6	0.8
1	3.5e-02	1.1e-01	2.3e-01	3.8e-01
2	5.4e-03	5.1e-02	9.3e-02	2.2e-01
4	1.2e-03	1.5e-02	3.6e-02	5.4e-02
8	6.7e-04	1.8e-03	8.7e-03	5.8e-02
16	5.1e-04	7.6e-02	8.4e-04	1.1e-03
32	7.4e-04	5.2e-04	4.2e-04	7.0e-04
64	4.5e-04	4.8e-04	1.2e-03	7.8e-04
100	5.1e-04	5.7e-04	1.8e-02	1.2e-03
500	4.1e-04	3.8e-04	4.2e-04	8.2e-04

Table 4: *Burgers' equation*: Relative final prediction error measured in the $\mathcal{L}_2$ norm for different number of Runge-Kutta stages q and time-step sizes Δt . Here, the network architecture is fixed to 4 hidden layers with 50 neurons in each layer.

3.1.1. Example (Allen-Cahn Equation)

This example aims to highlight the ability of the proposed discrete time models to handle different types of nonlinearity in the governing partial differential equation. To this end, let us consider the Allen-Cahn equation along with periodic boundary conditions

$$\begin{aligned}
u_t - 0.0001u_{xx} + 5u^3 - 5u &= 0, \quad x \in [-1, 1], \quad t \in [0, 1], \\
u(0, x) &= x^2 \cos(\pi x), \\
u(t, -1) &= u(t, 1), \\
u_x(t, -1) &= u_x(t, 1).
\end{aligned} \tag{13}$$

The Allen-Cahn equation is a well-known equation from the area of reaction-diffusion systems. It describes the process of phase separation in multi-component alloy systems, including order-disorder transitions. For the Allen-Cahn equation, the nonlinear operator in equation (9) is given by

$$\mathcal{N}[u^{n+c_j}] = -0.0001u_{xx}^{n+c_j} + 5(u^{n+c_j})^3 - 5u^{n+c_j},$$

and the shared parameters of the neural networks (10) and (11) can be learned by minimizing the sum of squared errors

$$SSE = SSE_n + SSE_b, \tag{14}$$

$q \backslash \Delta t$	0.2	0.4	0.6	0.8
1	3.5e-02	1.1e-01	2.3e-01	3.8e-01
2	5.4e-03	5.1e-02	9.3e-02	2.2e-01
4	1.2e-03	1.5e-02	3.6e-02	5.4e-02
8	6.7e-04	1.8e-03	8.7e-03	5.8e-02
16	5.1e-04	7.6e-02	8.4e-04	1.1e-03
32	7.4e-04	5.2e-04	4.2e-04	7.0e-04
64	4.5e-04	4.8e-04	1.2e-03	7.8e-04
100	5.1e-04	5.7e-04	1.8e-02	1.2e-03
500	4.1e-04	3.8e-04	4.2e-04	8.2e-04

表 4: 在不同的 Runge-Kutta 阶数 q 和时间步长 Δt 下, 以 $\mathcal{L}_2$ 范数测量的 Burgers' equation: 相对最终预测误差。这里, 网络架构固定为 4 个隐藏层, 每层 50 个神经元。

3.1.1. Example (Allen-Cahn Equation)

这个例子旨在突出所提出的离散时间模型处理控制偏微分方程中不同类型非线性的能力。为此, 我们考虑带有周期边界条件的 Allen-Cahn 方程

$$\begin{aligned}
u_t - 0.0001u_{xx} + 5u^3 - 5u &= 0, \quad x \in [-1, 1], \quad t \in [0, 1], \\
u(0, x) &= x^2 \cos(\pi x), \\
u(t, -1) &= u(t, 1), \\
u_x(t, -1) &= u_x(t, 1).
\end{aligned} \tag{13}$$

Allen-Cahn 方程是反应-扩散系统领域中一个著名的方程。它描述了多组分合金系统中的相分离过程, 包括有序-无序转变。对于 Allen-Cahn 方程, 方程 (9) 中的非线性算子由以下给出

$$\mathcal{N}[u^{n+c_j}] = -0.0001u_{xx}^{n+c_j} + 5(u^{n+c_j})^3 - 5u^{n+c_j},$$

神经网络 (10) 和 (11) 的共享参数可以通过最小化平方误差和来学习

$$SSE = SSE_n + SSE_b, \tag{14}$$

where

$$SSE_n = \sum_{j=1}^{q+1} \sum_{i=1}^{N_n} |u_j^n(x^{n,i}) - u^{n,i}|^2,$$

and

$$\begin{aligned} SSE_b &= \sum_{i=1}^q |u^{n+c_i}(-1) - u^{n+c_i}(1)|^2 + |u^{n+1}(-1) - u^{n+1}(1)|^2 \\ &+ \sum_{i=1}^q |u_x^{n+c_i}(-1) - u_x^{n+c_i}(1)|^2 + |u_x^{n+1}(-1) - u_x^{n+1}(1)|^2. \end{aligned}$$

Here, $\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ corresponds to the data at time t^n . We have generated a training and test data-set set by simulating the Allen-Cahn equation (13) using conventional spectral methods. Specifically, starting from an initial condition $u(0, x) = x^2 \cos(\pi x)$ and assuming periodic boundary conditions $u(t, -1) = u(t, 1)$ and $u_x(t, -1) = u_x(t, 1)$, we have integrated equation (13) up to a final time $t = 1.0$ using the Chebfun package [22] with a spectral Fourier discretization with 512 modes and a fourth-order explicit Runge-Kutta temporal integrator with time-step $\Delta t = 10^{-5}$.

In this example, we assume $N_n = 200$ initial data points that are randomly sub-sampled from the exact solution at time $t = 0.1$, and our goal is to predict the solution at time $t = 0.9$ using a single time-step with size $\Delta t = 0.8$. To this end, we employ a discrete time *physics informed neural network* with 4 hidden layers and 200 neurons per layer, while the output layer predicts 101 quantities of interest corresponding to the $q = 100$ Runge-Kutta stages $u^{n+c_i}(x)$, $i = 1, \dots, q$, and the solution at final time $u^{n+1}(x)$. Figure 4 summarizes our predictions after the network has been trained using the loss function of equation (14). Evidently, despite the complex dynamics leading to a solution with two sharp internal layers, we are able to obtain an accurate prediction of the solution at $t = 0.9$ using only a small number of scattered measurements at $t = 0.1$.

4. Summary and Discussion

We have introduced *physics informed neural networks*, a new class of universal function approximators that is capable of encoding any underlying

哪里

$$SSE_n = \sum_{j=1}^{q+1} \sum_{i=1}^{N_n} |u_j^n(x^{n,i}) - u^{n,i}|^2,$$

和

$$\begin{aligned} SSE_b &= \sum_{i=1}^q |u^{n+c_i}(-1) - u^{n+c_i}(1)|^2 + |u^{n+1}(-1) - u^{n+1}(1)|^2 \\ &+ \sum_{i=1}^q |u_x^{n+c_i}(-1) - u_x^{n+c_i}(1)|^2 + |u_x^{n+1}(-1) - u_x^{n+1}(1)|^2. \end{aligned}$$

这里, $\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ 对应于时间 t^n 的数据。我们通过使用传统谱方法模拟 Allen-Cahn 方程 (13) 生成了训练和测试数据集。具体来说, 从初始条件 $u(0, x) = x^2 \cos(\pi x)$ 开始, 并假设周期性边界条件 $u(t, -1) = u(t, 1)$ 和 $u_x(t, -1) = u_x(t, 1)$, 我们使用 Chebfun 软件包 [22] 对方程 (13) 进行了积分, 直到最终时间 $t = 1.0$, 采用具有 512 个模态的谱傅里叶离散化, 以及具有时间步长 $\Delta t = 10^{-5}$ 的四阶显式 Runge-Kutta 时间积分器。

在这个例子中, 我们假设有 $N_n = 200$ 个初始数据点, 这些数据点是从时间 $t = 0.1$ 的精确解中随机抽样的, 我们的目标是使用单步时间步长 $\Delta t = 0.8$ 来预测时间 $t = 0.9$ 的解。为此, 我们采用一个离散时间 *physics informed neural network*, 该网络具有 4 个隐藏层, 每层 200 个神经元, 而输出层预测对应于 $q = 100$ 个龙格-库塔阶段 $u^{n+c_i}(x)$ 、 $i = 1, \dots, q$ 的 101 个感兴趣量, 以及最终时间 $u^{n+1}(x)$ 的解。图 4 总结了网络在使用方程 (14) 的损失函数训练后得到的预测结果。显然, 尽管复杂的动态导致解具有两个尖锐的内部层, 我们仍然能够仅利用时间 $t = 0.1$ 的少量散乱测量准确预测时间 $t = 0.9$ 的解。

4. Summary and Discussion

我们引入了 *physics informed neural networks*, 一种新型的通用函数逼近器, 能够编码任何潜在的

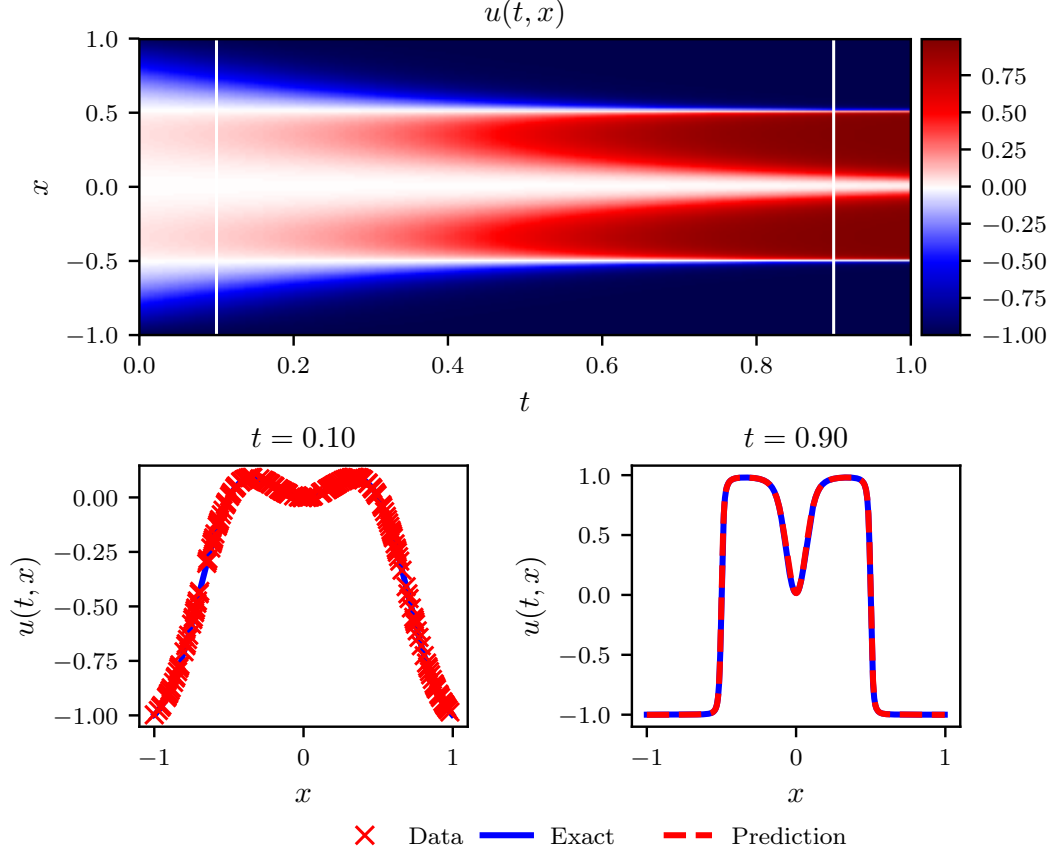


Figure 4: *Allen-Cahn equation*: *Top*: Solution $u(t, x)$ along with the location of the initial training snapshot at $t = 0.1$ and the final prediction snapshot at $t = 0.9$. *Bottom*: Initial training data and final prediction at the snapshots depicted by the white vertical lines in the top panel. The relative $\mathcal{L}_2$ error for this case is $6.99 \cdot 10^{-3}$.

physical laws that govern a given data-set, and can be described by partial differential equations. In this work, we design data-driven algorithms for inferring solutions to general nonlinear partial differential equations, and constructing computationally efficient physics-informed surrogate models. The resulting methods showcase a series of promising results for a diverse collection of problems in computational science, and open the path for endowing deep learning with the powerful capacity of mathematical physics to model the world around us. As deep learning technology is continuing to grow rapidly both in terms of methodological and algorithmic developments, we believe that this is a timely contribution that can benefit practitioners across

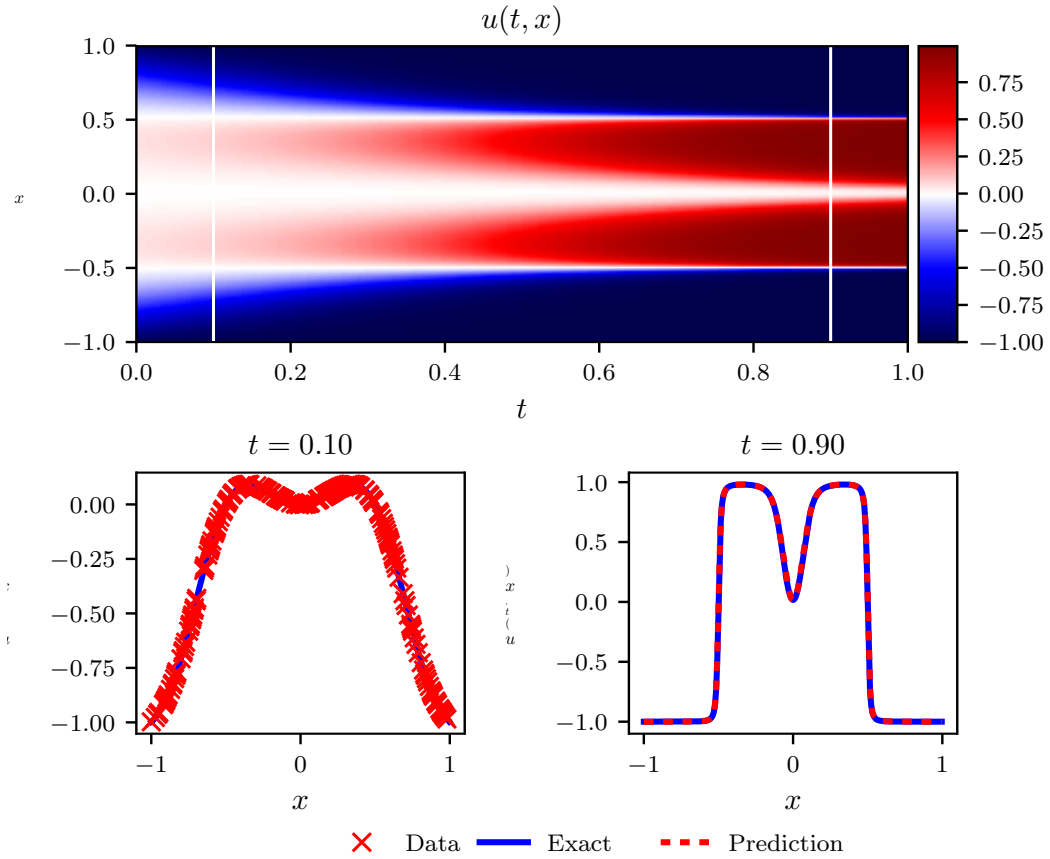


图 4: Allen-Cahn equation: Top: 解决方案 $u(t, x)$, 以及初始训练快照位于 $t = 0.1$ 和最终预测快照位于 $t = 0.9$ 的位置。Bottom: 顶部面板中白色垂直线所示的快照对应的初始训练数据和最终预测。该情况的相对 $\mathcal{L}_2$ 误差为 $6.99 \cdot 10^{-3}$ 。

支配给定数据集的物理定律，可以用偏微分方程描述。在这项工作中，我们设计了用于推断一般非线性偏微分方程解的数据驱动算法，并构建了计算上高效的物理知情替代模型。所得方法展示了一系列在计算科学中多样问题上的有前景的结果，并为赋予深度学习以数学物理的强大能力以模拟我们周围世界开辟了道路。随着深度学习技术在方法论和算法发展方面的快速增长，我们认为这是一个及时的贡献，可以惠及各类从业者。

a wide range of scientific domains. Specific applications that can readily enjoy these benefits include, but are not limited to, data-driven forecasting of physical processes, model predictive control, multi-physics/multi-scale modeling and simulation.

We must note however that the proposed methods should not be viewed as replacements of classical numerical methods for solving partial differential equations (e.g., finite elements, spectral methods, etc.). Such methods have matured over the last 50 years and, in many cases, meet the robustness and computational efficiency standards required in practice. Our message here, as advocated in Section 3, is that classical methods such as the Runge-Kutta time-stepping schemes can coexist in harmony with deep neural networks, and offer invaluable intuition in constructing structured predictive algorithms. Moreover, the implementation simplicity of the latter greatly favors rapid development and testing of new ideas, potentially opening the path for a new era in data-driven scientific computing. This will be further highlighted in the second part of this paper in which *physics informed neural networks* are put to the test of data-driven discovery of partial differential equations.

Finally, in terms of future work, one pressing question involves addressing the problem of quantifying the uncertainty associated with the neural network predictions. Although this important element was naturally addressed in previous work employing Gaussian processes [9], it not captured by the proposed methodology in its present form and requires further investigation.

Acknowledgements

This work received support by the DARPA EQUiPS grant N66001-15-2-4055, the MURI/ARO grant W911NF-15-1-0562, and the AFOSR grant FA9550-17-1-0013. All data and codes used in this manuscript are publicly available on GitHub at <https://github.com/maziarraissi/PINNs>.

References

- [1] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, in: Advances in neural information processing systems, pp. 1097–1105.

广泛的科学领域。可以直接享受这些好处的具体应用包括但不限于物理过程的数据驱动预测、模型预测控制、多物理/多尺度建模与仿真。

然而，我们必须注意，所提出的方法不应被视为传统求解偏微分方程的数值方法（例如有限元法、谱方法等）的替代品。这些方法在过去50年中已经成熟，并且在许多情况下能够满足实际所需的稳健性和计算效率标准。正如第3节中所倡导的，我们这里的观点是，诸如Runge-Kutta时间步进方案之类的传统方法可以与深度神经网络和谐共存，并在构建结构化预测算法时提供宝贵的直觉。此外，后者实现的简便性极大地有利于新想法的快速开发和测试，可能为数据驱动的科学计算开辟新的时代。在本文的第二部分中，这一点将得到进一步强调，其中*physics informed neural networks*将接受基于数据的偏微分方程发现的检验。

最后，在未来的工作方面，一个紧迫的问题是解决量化神经网络预测不确定性的问题。虽然这一重要元素在先前使用高斯过程的工作中得到了自然的处理 [9]，但在当前形式下所提出的方法中并未被捕捉到，因此需要进一步研究。

Acknowledgements

本工作得到了DARPA EQUiPS资助项目N66001-15-2-4055、MURI/ARO资助项目W911NF-15-1-0562以及AFOSR资助项目FA9550-17-1-0013的支持。本文使用的所有数据和代码均可在GitHub上公开获取，地址为<https://github.com/maziarraissi/PINNs>。

References

- [1] A. Krizhevsky, I. Sutskever, G. E. Hinton, 使用深度卷积神经网络进行Imagenet分类，载于：神经信息处理系统进展，第1097–1105页。

- [2] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (2015) 436–444.
- [3] B. M. Lake, R. Salakhutdinov, J. B. Tenenbaum, Human-level concept learning through probabilistic program induction, *Science* 350 (2015) 1332–1338.
- [4] B. Alipanahi, A. Delong, M. T. Weirauch, B. J. Frey, Predicting the sequence specificities of DNA-and RNA-binding proteins by deep learning, *Nature biotechnology* 33 (2015) 831–838.
- [5] M. Raissi, P. Perdikaris, G. E. Karniadakis, Inferring solutions of differential equations using noisy multi-fidelity data, *Journal of Computational Physics* 335 (2017) 736–746.
- [6] M. Raissi, P. Perdikaris, G. E. Karniadakis, Machine learning of linear differential equations using Gaussian processes, *Journal of Computational Physics* 348 (2017) 683 – 693.
- [7] H. Owhadi, Bayesian numerical homogenization, *Multiscale Modeling & Simulation* 13 (2015) 812–828.
- [8] C. E. Rasmussen, C. K. Williams, Gaussian processes for machine learning, volume 1, MIT press Cambridge, 2006.
- [9] M. Raissi, P. Perdikaris, G. E. Karniadakis, Numerical Gaussian processes for time-dependent and non-linear partial differential equations, arXiv preprint arXiv:1703.10230 (2017).
- [10] M. Raissi, G. E. Karniadakis, Hidden physics models: Machine learning of nonlinear partial differential equations, arXiv preprint arXiv:1708.00588 (2017).
- [11] H. Owhadi, C. Scovel, T. Sullivan, et al., Brittleness of Bayesian inference under finite information in a continuous world, *Electronic Journal of Statistics* 9 (2015) 1–79.
- [12] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, *Neural networks* 2 (1989) 359–366.

- [2] Y. LeCun, Y. Bengio, G. Hinton, 深度学习, 自然 521 (2015) 436–444.
- [3] B. M. Lake, R. Salakhutdinov, J. B. Tenenbaum, 通过概率程序归纳实现人类水平的概念学习, 《科学》 350 (2015) 1332–1338。
- [4] B. Alipanahi, A. Delong, M. T. Weirauch, B. J. Frey, 通过深度学习预测DNA和RNA结合蛋白的序列特异性, 《自然生物技术》 33 (2015) 831–838。
- [5] M. Raissi, P. Perdikaris, G. E. Karniadakis, 使用噪声多精度数据推断微分方程的解, 计算物理学报 335 (2017) 736–746.
- [6] M. Raissi, P. Perdikaris, G. E. Karniadakis, 使用高斯过程进行线性微分方程的机器学习, 《计算物理学杂志》 348 (2017) 683–693。
- [7] H. Owhadi, 贝叶斯数值均匀化, 《多尺度建模与仿真》 13 (2015) 812–828.
- [8] C. E. Rasmussen, C. K. Williams, 《用于机器学习的高斯过程》, 第1卷, 麻省理工学院出版社, 剑桥, 2006年。
- [9] M. Raissi, P. Perdikaris, G. E. Karniadakis, 用于时变和非线性偏微分方程的数值高斯过程, arXiv 预印本 arXiv:1703.10230 (2017).
- [10] M. Raissi, G. E. Karniadakis, 隐藏物理模型: 非线性偏微分方程的机器学习, arXiv 预印本 arXiv:1708.00588 (2017).
- [11] H. Owhadi, C. Scovel, T. Sullivan 等, 连续世界中有限信息下贝叶斯推理的脆弱性, 《电子统计学杂志》 9 (2015) 1–79。
- [12] K. Hornik, M. Stinchcombe, H. White, 多层前馈网络是通用逼近器, 神经网络 2 (1989) 359–366.

- [13] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, Automatic differentiation in machine learning: a survey, arXiv preprint arXiv:1502.05767 (2015).
- [14] C. Basdevant, M. Deville, P. Haldenwang, J. Lacroix, J. Ouazzani, R. Peyret, P. Orlandi, A. Patera, Spectral and finite difference solutions of the Burgers equation, *Computers & fluids* 14 (1986) 23–41.
- [15] S. H. Rudy, S. L. Brunton, J. L. Proctor, J. N. Kutz, Data-driven discovery of partial differential equations, *Science Advances* 3 (2017).
- [16] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, et al., Tensorflow: Large-scale machine learning on heterogeneous distributed systems, arXiv preprint arXiv:1603.04467 (2016).
- [17] D. C. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization, *Mathematical programming* 45 (1989) 503–528.
- [18] I. Goodfellow, Y. Bengio, A. Courville, *Deep learning*, MIT press, 2016.
- [19] D. Kingma, J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980 (2014).
- [20] A. Choromanska, M. Henaff, M. Mathieu, G. B. Arous, Y. LeCun, The loss surfaces of multilayer networks, in: *Artificial Intelligence and Statistics*, pp. 192–204.
- [21] R. Shwartz-Ziv, N. Tishby, Opening the black box of deep neural networks via information, arXiv preprint arXiv:1703.00810 (2017).
- [22] T. A. Driscoll, N. Hale, L. N. Trefethen, *Chebfun guide*, 2014.
- [23] M. Stein, Large sample properties of simulations using latin hypercube sampling, *Technometrics* 29 (1987) 143–151.
- [24] A. Iserles, *A first course in the numerical analysis of differential equations*, 44, Cambridge University Press, 2009.

[13] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, 机器学习中的自动微分：综述, arXiv 预印本 arXiv:1502.05767 (2015).

[14] C. Basdevant, M. Deville, P. Haldenwang, J. Lacroix, J. Ouazzani, R. Peyret, P. Orlandi, A. Patera, 伯格斯方程的谱方法和有限差分解法, 计算与流体 14 (1986) 23–41.

[15] S. H. Rudy, S. L. Brunton, J. L. Proctor, J. N. Kutz, 基于数据的偏微分方程发现, 科学进展 3 (2017)。 [16] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin 等, TensorFlow：异构分布式系统上的大规模机器学习, arXiv 预印本 arXiv:1603.04467 (2016)。 [17] D. C. Liu, J. Nocedal, 关于大规模优化的有限内存 BFGS 方法, 数学规划 45 (1989) 503–528。 [18] I. Goodfellow, Y. Bengio, A. Courville, 深度学习, MIT 出版社, 2016。 [19] D. Kingma, J. Ba, Adam：一种随机优化方法, arXiv 预印本 arXiv:1412.6980 (2014)。

[20] A. Choromanska, M. Henaff, M. Mathieu, G. B. Arous, Y. LeCun, 多层网络的损失表面, 载于：人工智能与统计, 第192–204页。

[21] R. Shwartz-Ziv, N. Tishby, 通过信息打开深度神经网络的黑箱, arXiv 预印本 arXiv:1703.00810 (2017)。 [22] T. A. Driscoll, N. Hale, L. N. Trefethen, Chebfun 指南, 2014。 [23] M. Stein, 使用拉丁超立方抽样的模拟的大样本性质, Technometrics 29 (1987) 143–151。 [24] A. Iserles, 微分方程数值分析初阶课程, 44, 剑桥大学出版社, 2009。